University Mohammed VI Polytechnic

College of Computing

Lab 6: SQL – DDL and DML

Professor: Karima Echihabi
Program: Computer Engineering
Session: Fall 2026 – UM6P – CC

Preparatory Exercises

Before starting Project Part VI, students should review and solve the following textbook exercises.

Textbook Exercises

Reference: Ramakrishnan, Raghu, and Johannes Gehrke, *Database Management Systems*, 3rd Ed. [1].

- Chapter 5 (SQL: Queries, Constraints, Triggers): Exercise 5.3

Reference Documentation

All labs use **MySQL 8.0**. Keep the reference manual open while you work.

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- `CREATE TABLE` and constraints: https://dev.mysql.com/doc/refman/8.0/en/create-table.html
- DML statements (`INSERT`, `UPDATE`, `DELETE`, `SELECT`): https://dev.mysql.com/doc/refman/8.0/en/sql-data-manipulation-statements.html
- MySQL data types: https://dev.mysql.com/doc/refman/8.0/en/data-types.html

When a query misbehaves, read the error message first, then the manual page of the statement.

Objectives

- Implement the final RLMS relational schema using SQL Data Definition Language (DDL).

- Populate and modify the RLMS database using SQL Data Manipulation Language (DML).

  Retrieve information using `SELECT` queries, which are part of the DML, progressing from simple single-table queries to joins and grouped aggregations.

Tasks

Part 1: Data Definition Language (DDL)

Using the final RLMS relational schema obtained in the previous labs, with the following revision to the People section, create the corresponding SQL tables.

Revised People schema (adopted from Lab 6 onward)

For the SQL labs we adopt a slightly revised People schema. It stores the same information as the Lab 2 design but is easier to implement and query:

- `Person(pid, firstName, lastName, email, affiliation, startDate, endDate°)` and `Student(pid, program)` — unchanged;
- `Employee(pid, phone, office, position, supervisorPid°)` — `position` is now stored here for every employee (dean, faculty, scientist, postdoc, secretary, technician, . . .): its values are managed in one place, and listing every employee with their position is a single-table query;
- `Academic(pid)`, `Administrative(pid)`, `Technical(pid)`, `Faculty(pid)` — kept as plain subtype tables because foreign keys target them (`Supervises` → `Faculty`; `Manages` and `Advises` → `Academic`; `DoneBy` and `Monitors` → `Technical`) or because membership must be testable (`Administrative`, for the participation and reservation exclusions). `Faculty` references `Academic`; `Administrative` and `Technical` now reference `Employee` directly;
- `NonAcademic` is dropped: it carries no attributes and no relationship references it.

All other relations are unchanged from the Lab 2 solution.

Your implementation must include:

- appropriate SQL data types;
- primary keys;
- composite primary keys whenever required (e.g., the weak entities `Maintenance` and `CalibrationRecord` are identified by the owning equipment unit together with their partial key, per MA2 and CA4);
- foreign keys;
- a nullable foreign key on `Reservation` referencing the person who approved it, since only some reservations require approval and the application must keep track of who approved them (RE3);

- `NOT NULL` constraints where required by the schema;
- optionally, `UNIQUE` constraints for attributes you assume to be candidate keys (e.g., a person's email address); the requirements do not impose such uniqueness, so any `UNIQUE` you add must be stated as an assumption in your report;
- `DEFAULT` values where sensible (e.g., a new reservation starts with status `'Pending'`);
- `ON DELETE` / `ON UPDATE` referential actions, choosing and justifying the appropriate action for each foreign key (e.g., `CASCADE` from `Reservation` to `Reserves`, `SET NULL` for the approver).

The implementation should include the relations of the final RLMS schema, in particular the ternary `Supplies` relation (one negotiated unit price per supplier–consumable–laboratory combination, CO4), the `Stocks` relation with its monitoring assignment (CO6–CO8), and the `Consumes` relation linking each reservation to the laboratory stocks from which consumables are taken, i.e., referencing `Stocks(labId, consId)` (CO9–CO10).

Part 2: Data Manipulation Language (DML) – Population and Updates

Populate the RLMS database with enough data to meaningfully test the queries in Part 3. Keep the dataset small: three to five rows per table are sufficient, provided every category below has at least one example. Your dataset should contain examples of:

- persons belonging to different subtypes;
- several laboratories located in different buildings;
- research projects with different participants and roles;
- budgets managed by academic employees and funding both laboratories and research projects;
- several equipment models and physical equipment units;
- certifications held by several persons, and at least one equipment model requiring one or more certifications;

  reservations covering the different statuses (Pending, Approved, Rejected, Cancelled, Completed), where each approved reservation records its approver;
- reservations involving one or more equipment units, possibly belonging to different equipment models;
- maintenance and calibration records;
- consumables stocked in different laboratories;
- reservations that consume consumables, with the quantity used recorded;
- for each laboratory–consumable stock, the technical staff member monitoring it and the date the monitoring assignment began;

- multiple suppliers and supply arrangements, including at least one arrangement for a consumable that the laboratory does not currently keep in stock;
- the data needed to replay the Lab 3 queries and to prepare Lab 7: a building named `'Riad5'`, a laboratory named `'Physics Laboratory'`, equipment model categories `'Optical Microscope'` and `'Thermometer'`, at least one approved reservation whose planned start falls within the next fourteen days, one person and one research project with no reservations, one equipment model with no units, and one stock at or below its reorder threshold.

Then perform the following DML operations:

1. Insert at least one new `Person` and the corresponding subtype record.
2. Insert a new `EquipmentUnit` associated with an existing `EquipmentModel` and `Laboratory`.
3. Insert a new `Reservation` together with at least one corresponding tuple in `Reserves`.
4. Update the status of one reservation from `'Pending'` to `'Approved'`, recording the person who approved it.
5. Update the quantity of one laboratory–consumable stock occurrence.
6. Insert a tuple recording that an approved reservation consumed a given quantity of a consumable.
7. Using an `UPDATE` with a nested subquery, cancel all pending reservations made for research projects whose status is `'Cancelled'`.
8. Delete one test tuple of your choice without violating referential integrity.

Part 3: Data Manipulation Language (DML) – Queries

Write and execute SQL queries for the following questions.

The queries are ordered approximately from simpler single-table queries to joins and grouped aggregations.

A. Simple Queries

1. List all persons, showing their identifier, first name, last name, and email address, ordered by last name and then first name.
2. List all distinct disciplines represented by the laboratories.
3. List all persons whose email address ends with `'@um6p.ma'`.
4. List all currently active persons, i.e., persons whose end date is not set.
5. List all reservations whose planned start time falls between two given timestamps (e.g., the month of November 2026).

B. Queries Using Joins

6. List all participants in research projects, showing the person's full name, project title, and participation role.

7. List each budget together with the full name of the academic employee who manages it.

8. List every reservation together with the full name of the person who made it and the title of the associated research project.

9. List the equipment units included in each reservation. Display the reservation identifier, equipment serial number, equipment model commercial name, and laboratory name.

10. List all approved reservations together with the full name of the person who approved them.

11. For each laboratory–consumable stock, display the laboratory name, the consumable name, the quantity on hand, and the full name of the Technical staff member monitoring this stock.

C. Aggregation Queries

12. Count the number of equipment units in each laboratory.

13. For each consumable, return the total quantity used across all reservations.

D. Queries Combining Joins and Aggregation

14. List laboratories containing more than three equipment units. Display the laboratory name and the number of equipment units.

15. For each research project, return the project title and the total amount granted by the budgets funding it.

16. For each laboratory and equipment category, return the number of equipment units and their average purchase cost.

17. For each supplier, count the number of distinct consumables that the supplier can provide across all laboratories.

18. For each laboratory and consumable, return the lowest unit price negotiated across all suppliers supplying that consumable to that laboratory.

19. For each reservation that consumed at least one consumable, return the reservation identifier, the full name of the person who made it, and the total quantity of consumables used.

E. Set Operations and Nested Queries

20. Using a set operation, list the identifiers and full names of persons who both supervise a laboratory and manage a budget.

21. List the persons who have never made any reservation (use `EXCEPT` or `NOT IN`).

22. List the equipment units that have never been reserved (use `NOT EXISTS`).

23. Using `ALL`, list the equipment unit(s) whose purchase cost is greater than or equal to that of every equipment unit in the database.

F. Division Queries

24. For each equipment model that requires at least one certification, list the persons who hold ALL the certifications required by that model.

Part 4: Open Question

While writing your DDL script, you expressed some integrity rules with primary keys, foreign keys, and `NOT NULL` constraints. Other requirements of the RLMS application, however, cannot be enforced with the SQL constructs covered so far.

Identify at least three such requirements, citing their requirement codes. For each one, explain briefly (2–3 sentences) why the SQL learned so far cannot enforce it: what would the database need to check, and at what moment? You are not expected to implement a solution — these mechanisms (triggers and advanced constraints) are the subject of the next lab.

Deliverable

- A SQL DDL script implementing the final RLMS relational schema, including primary keys, foreign keys, and the required constraints.
- A single SQL DML script containing the population data, the modification operations of Part 2, and all queries of Part 3, in that order.
- Executed outputs for all DML operations and queries in the form of logs.
- A short report (up to two pages) that: 1) describes the role of each team member; 2) summarizes the main implementation difficulties encountered; 3) explains any assumptions made while implementing or populating the database; and 4) answers the open question of Part 4.
- The deliverable is due on Sunday 01/11 at 23:59.

References

[1] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.